ENTREGA 2: SISTEMAS EMBARCADOS E ROBÓTICA

1. Introdução e visão geral

O Smart Tennis Arena é um sistema de treinamento gamificado de tênis que utiliza sensores piezoelétricos para detectar impactos em áreas específicas de um painel. O objetivo é transformar exercícios de precisão em desafios interativos: o sistema indica um alvo, detecta a resposta do jogador e fornece feedback visual e sonoro.

O Arduino Uno R3 atua como unidade central de processamento. Ele recebe os sinais dos sensores, executa a lógica do treinamento, controla indicadores e o buzzer e troca informações com um computador por meio da comunicação serial USB.

A arquitetura descrita considera cinco alvos, cada um associado a um sensor piezoelétrico. A documentação também prevê a expansão das saídas com registradores 74HC595 para controlar conjuntos de LEDs usando poucos pinos do Arduino.

1.1. Componentes e funções

Componente	Função
Arduino Uno R3	Controla a aquisição dos sensores, a lógica do treinamento e os periféricos.
5 sensores piezoelétricos	Transformam vibrações e impactos mecânicos em sinais elétricos.
LEDs	Indicam visualmente os alvos e os estados do treinamento.
Resistores de 1 MΩ	Oferecem caminho para descarga da carga elétrica dos piezos.
Diodos Zener de 5,1 V	Limitam picos de tensão e ajudam a proteger as entradas analógicas no circuito físico.
Buzzer	Emite sinais sonoros de inicialização, acerto, erro e emergência.
Botão de emergência	Permite solicitar a interrupção de emergência.
Protoboard e jumpers	Permitem montar e interligar os componentes.
Cabo USB	Fornece alimentação ao Arduino e comunicação serial com o computador.
74HC595	Expande as saídas digitais; cada registrador disponibiliza oito saídas controladas serialmente.

2. Conexões eletrônicas

2.1. Sensores piezoelétricos

O piezoelétrico converte deformação mecânica em sinal elétrico. Quando o jogador atinge uma placa do painel, a vibração chega ao sensor, que gera uma variação de tensão detectável pelo Arduino.

Sensor	Entrada analógica
Piezo 1 — Alvo 1	A0
Piezo 2 — Alvo 2	A1
Piezo 3 — Alvo 3	A2
Piezo 4 — Alvo 4	A3

Piezo 5 — Alvo 5	A4

Cada piezo possui dois terminais: um é ligado à entrada analógica correspondente e o outro ao GND. O resistor de 1 MΩ é instalado em paralelo com o sensor, entre os mesmos dois pontos elétricos.

O resistor oferece um caminho para descarregar a carga que pode permanecer no sensor após o impacto. Na montagem física, a especificação também prevê um diodo Zener de 5,1 V para limitar picos de tensão. O resistor, sozinho, não substitui essa proteção.

2.2. LEDs e resistores

Os LEDs fornecem indicação visual. Na configuração de controle direto apresentada no firmware, os cinco LEDs de alvo estão associados aos pinos D3, D4, D5, D6 e D7. Cada LED deve utilizar um resistor limitador de corrente adequado.

Alvo	Pino no firmware de controle direto
Alvo 1	D3
Alvo 2	D4
Alvo 3	D5
Alvo 4	D6
Alvo 5	D7

Quando o pino correspondente recebe HIGH, o LED acende; quando recebe LOW, o LED apaga. Essa tabela representa o modo direto definido no firmware enviado, não a configuração expandida de oito LEDs por alvo com 74HC595.

2.3. Registradores 74HC595

O 74HC595 é um registrador de deslocamento com oito saídas paralelas. Ele permite controlar oito LEDs usando três sinais principais do Arduino: DATA (dados), CLOCK (relógio de deslocamento) e LATCH (atualização das saídas).

Quando vários 74HC595 são encadeados, a saída serial de um registrador alimenta a entrada serial do próximo. Os sinais de clock e latch são compartilhados. Para cinco registradores, o Arduino envia cinco bytes em sequência, correspondentes a 40 saídas.

Importante: o firmware Smart Tennis fornecido ainda não está integrado a essa arquitetura de cinco registradores. USE_74HC595 está como false e as funções de LEDs utilizam os pinos diretos D3 a D7.

2.5. Botão de emergência

```
const uint8_t PIN_ESTOP_BUTTON = 2;
pinMode(PIN_ESTOP_BUTTON, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(PIN_ESTOP_BUTTON),
                handleEmergencyInterrupt, FALLING);
```

O botão é ligado entre D2 e GND. Com INPUT_PULLUP, a entrada normalmente permanece em HIGH e passa para LOW quando o botão é pressionado. A interrupção FALLING detecta essa transição.

O firmware sinaliza a emergência ao programa principal. Isso é uma parada lógica; se houver motores ou máquina de bolas, a interrupção física da energia dos atuadores exige também um circuito de segurança apropriado.

3. Código de teste dos sensores piezoelétricos

O Código de Teste do Piezo é um programa de diagnóstico. Seu objetivo é verificar se os cinco sensores respondem aos impactos e se o Arduino consegue identificar qual alvo foi atingido. Ele não implementa o treinamento completo.

3.1. Vetor de sensores

```
const int PIEZO[5] = { A0, A1, A2, A3, A4 };
```

O vetor PIEZO reúne os cinco pinos analógicos. O índice 0 representa A0, o índice 1 representa A1 e assim sucessivamente até A4. Isso permite percorrer os sensores com um único laço for.

3.2. Delta mínimo

```
const int DELTA_MINIMO = 20;
```

O Arduino Uno converte a leitura analógica em um valor entre 0 e 1023. O código compara a leitura atual com a anterior. O delta é a diferença entre essas duas leituras:

Delta = leitura atual − leitura anterior

Se a diferença for maior que 20, o sinal é considerado uma possível variação de impacto. Por exemplo, uma mudança de 100 para 125 produz delta 25 e satisfaz o limite; uma mudança de 100 para 115 produz delta 15 e não satisfaz.

Como o teste exige delta positivo, ele procura aumentos repentinos na leitura. Variações negativas não disparam a condição.

3.3. Debounce e memória de leitura

```
const unsigned long TEMPO_DEBOUNCE = 120;
unsigned long tempoUltimoImpacto[5] = {0, 0, 0, 0, 0};
int leituraAnterior[5] = {0, 0, 0, 0, 0};
```

O debounce estabelece um intervalo mínimo de 120 milissegundos entre detecções válidas do mesmo sensor. Isso ajuda a evitar que as oscilações de um único impacto sejam interpretadas como várias batidas.

tempoUltimoImpacto mantém um horário independente para cada sensor. leituraAnterior armazena a leitura mais recente de cada piezo, permitindo calcular o delta na próxima passagem do loop.

3.4. Inicialização — setup()

```
void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 5; i++) {
    pinMode(PIEZO[i], INPUT);
    leituraAnterior[i] = analogRead(PIEZO[i]);
  }
}
```

setup() é executada uma vez na inicialização. Serial.begin(115200) inicia a comunicação com o computador a 115200 bits por segundo. O laço for configura os cinco pinos como entradas e realiza uma primeira leitura para estabelecer a referência inicial.

3.5. Monitoramento contínuo — loop()

```
unsigned long tempoAtual = millis();
for (int i = 0; i < 5; i++) {
  int leituraAtual = analogRead(PIEZO[i]);
  int delta = leituraAtual - leituraAnterior[i];
  if (delta > DELTA_MINIMO &&
      (tempoAtual - tempoUltimoImpacto[i] >= TEMPO_DEBOUNCE)) {
    tempoUltimoImpacto[i] = tempoAtual;
    // Exibe o impacto no Monitor Serial
  }
  leituraAnterior[i] = leituraAtual;
}
```

millis() informa quantos milissegundos se passaram desde que o Arduino começou a executar o programa. O loop percorre os cinco sensores, lê cada um, calcula o delta e verifica duas condições: delta maior que o mínimo e tempo de debounce cumprido.

Quando as condições são verdadeiras, o código atualiza o horário do último impacto e imprime o número do alvo, a variação (delta) e a leitura absoluta. Ao final de cada repetição, atualiza leituraAnterior para que a próxima comparação use a amostra mais recente.

3.6. Exemplo de saída

```
-> [ALVO 1] Impacto! Variação (Delta): 45 | Leitura Absoluta: 180
-> [ALVO 3] Impacto! Variação (Delta): 32 | Leitura Absoluta: 145
```

A saída informa qual alvo foi detectado, o tamanho da variação do sinal e o valor absoluto lido pelo conversor analógico-digital.

4. Firmware principal Smart Tennis

O firmware Smart Tennis implementa a lógica de operação do protótipo. Além de ler os sensores, organiza o sistema em estados, ativa alvos, controla o tempo dos desafios, gera feedback sonoro e visual, recebe comandos do computador e envia eventos estruturados pela serial.

4.1. Definição dos pinos e parâmetros

```
#define NUM_TARGETS 5
const uint8_t PIN_PIEZO[NUM_TARGETS] = {A0, A1, A2, A3, A4};
const uint8_t PIN_LEDS[NUM_TARGETS] = {3, 4, 5, 6, 7};
const uint8_t PIN_BUZZER = 8;
const uint8_t PIN_ESTOP_BUTTON = 2;
const uint8_t PIN_STATUS_LED = 13;
```

NUM_TARGETS define a quantidade de alvos. PIN_PIEZO associa cada sensor a uma entrada analógica. PIN_LEDS associa cada alvo a um pino digital no modo direto. As demais constantes identificam o buzzer, o botão de emergência e o LED de status.

4.2. Limiar de detecção

```
uint16_t piezoThresholds[NUM_TARGETS] = {100, 100, 100, 100, 100};
```

No firmware principal, a detecção utiliza um limiar absoluto. Uma leitura igual ou superior ao limite do sensor é considerada um impacto. O vetor permite ajustar cada alvo separadamente. O valor inicial 100 é uma configuração de partida e deve ser calibrado com os sensores e a estrutura física reais.

4.3. Estados do firmware

```
enum FirmwareState {
  STATE_BOOT,
  STATE_IDLE_READY,
  STATE_TARGET_ACTIVE,
  STATE_PAUSED,
  STATE_EMERGENCY_STOP
};
```

Estado	Significado
STATE_BOOT	Inicialização do equipamento.
STATE_IDLE_READY	Sistema pronto, aguardando comando ou monitorando sensores em repouso.
STATE_TARGET_ACTIVE	Um alvo está ativo e o sistema aguarda o impacto.
STATE_PAUSED	Treinamento pausado.
STATE_EMERGENCY_STOP	Parada de emergência acionada.

A máquina de estados organiza o comportamento do sistema. O firmware decide o que fazer de acordo com o estado atual.

4.4. Inicialização — setup()

Na inicialização, o firmware abre a comunicação serial, configura LEDs, buzzer, sensores e botão de emergência. Em seguida, realiza um autoteste: acende rapidamente cada LED de alvo, apaga-o, limpa as saídas e emite um sinal sonoro de pronto. Por fim, muda para STATE_IDLE_READY e envia um evento READY ao computador.

4.5. Laço principal — loop()

O loop verifica primeiro se uma emergência foi sinalizada. Se isso acontecer, muda para STATE_EMERGENCY_STOP, desativa o alvo ativo, apaga os LEDs, toca o alarme e envia ESTOP_ALERT. Depois, processa os comandos recebidos pela serial.

Se estiver em STATE_TARGET_ACTIVE, o firmware verifica se o tempo do alvo terminou. Se terminou, registra MISS; se ainda há tempo, chama a rotina de verificação dos sensores. Em STATE_IDLE_READY, chama a rotina de monitoramento livre dos sensores.

4.6. Leitura estabilizada

```
uint16_t readPiezoStable(uint8_t index) {
  analogRead(PIN_PIEZO[index]);
  delayMicroseconds(20);
  return analogRead(PIN_PIEZO[index]);
}
```

A função realiza uma leitura descartada, aguarda 20 microssegundos e então realiza a leitura utilizada. A intenção é permitir a acomodação do circuito de amostragem do conversor ADC ao alternar entre canais analógicos.

4.7. Acerto, erro e tentativa não concluída

Durante um desafio, o firmware prioriza a leitura do sensor correspondente ao alvo ativo. Se esse sensor atingir o limiar configurado, o impacto é classificado como HIT: o alvo é desativado, os LEDs são apagados, o buzzer toca e um evento é enviado pela serial.

Se o alvo ativo não disparar, o programa verifica os outros sensores. Para reduzir falsos positivos causados por vibração transmitida pela estrutura, aplica aos alvos não esperados um limiar 25% maior que o limiar individual.

```
uint16_t wrongThreshold = (piezoThresholds[i] * 5) / 4;
```

Se outro sensor ultrapassar esse limite, o firmware registra WRONG_HIT, informando o alvo atingido e o alvo esperado. Se o tempo do desafio terminar sem acerto, o firmware registra MISS.

4.8. Feedback sonoro

Situação	Comportamento sonoro no código
Inicialização / pronto	Dois tons curtos em frequências diferentes.
Acerto	Sequência de dois tons ascendentes.
Alvo incorreto	Tom mais grave.
Tempo esgotado	Tom de aviso.
Emergência	Repetição de três tons de alarme.

Os tons são gerados por tone(), que recebe o pino do buzzer, a frequência e a duração em milissegundos.

5. Comunicação serial com a aplicação

A comunicação serial USB permite que o computador envie comandos ao Arduino e receba eventos produzidos pelo firmware. O programa lê os caracteres recebidos até encontrar uma quebra de linha e então encaminha a mensagem para executeCommand().

5.1. Comandos recebidos

Comando	Função prevista no firmware
START	Inicia um desafio informando o número do alvo e a duração.
PAUSE	Pausa o desafio ativo e guarda o tempo restante.
RESUME	Retoma um desafio pausado.
RESET	Retorna ao estado pronto e limpa o alvo ativo.
PING	Solicita uma resposta de comunicação.
FEED	Registra a solicitação de acionamento da máquina de bolas e envia confirmação de despacho.
BUZZ	Aciona o buzzer com frequência e duração informadas.
SET_LEDS	Acende todos os LEDs ou um alvo específico, conforme o parâmetro.
CALIB	Atualiza os limiares dos sensores a partir de uma lista de valores.
POLL	Solicita as leituras instantâneas dos cinco sensores.
ESTOP	Aciona a parada de emergência lógica.

Exemplo de comando START:

```
{"cmd":"START","tgt":2,"time":5000}
```

O comando solicita a ativação do alvo 2 durante 5.000 milissegundos, equivalentes a cinco segundos.

5.2. Eventos enviados

Evento	Significado
READY	O equipamento terminou a inicialização e está pronto.
HIT	O alvo esperado foi atingido.
WRONG_HIT	Um alvo diferente do esperado recebeu impacto.
MISS	O tempo terminou sem acerto.
TAP	Toque detectado durante o monitoramento em repouso.
ESTOP_ALERT	A parada de emergência foi acionada.
PONG	Resposta ao comando PING.
ACK	Confirmação de processamento de determinados comandos.

Exemplo de evento HIT:

```
{"evt":"HIT","tgt":2,"val":180,"ts":12500}
```

evt identifica o evento; tgt indica o alvo; val informa a leitura do sensor; ts registra o tempo em milissegundos desde a inicialização do Arduino. A aplicação externa pode usar esses dados para atualizar o placar, registrar o desempenho e apresentar resultados.

6. Comparação entre os dois códigos

Característica	Teste do piezo	Firmware Smart Tennis
Objetivo	Diagnosticar os cinco sensores.	Controlar o sistema de treinamento.
Método de detecção	Variação positiva entre leituras (delta).	Limiar absoluto por sensor.
Parâmetro inicial	Delta mínimo de 20.	Limiar inicial de 100.
Debounce	120 ms por sensor.	Bloqueio global de 180 ms.
LEDs	Não controla.	Controla indicadores no modo direto.
Buzzer	Não controla.	Fornece feedback sonoro.
Emergência	Não implementada.	Implementada no firmware.
Comunicação	Mensagens de diagnóstico.	Comandos e eventos estruturados.

Os dois programas são independentes: apenas um deles é carregado no Arduino por vez. O programa de teste verifica os sensores; o firmware principal implementa a lógica integrada do Smart Tennis.

7. Pontos de integração e cuidados técnicos

7.1. Controle dos LEDs por 74HC595 ainda não integrado

No firmware enviado, USE_74HC595 está definido como false. Por isso, o programa configura os pinos D3 a D7 como saídas diretas para os cinco LEDs de alvo. A função update74HC595() envia apenas um byte e não está conectada à lógica setTargetLeds() / clearAllLeds().

Para utilizar cinco registradores encadeados e controlar 40 LEDs, será necessário adaptar o firmware para enviar cinco bytes por atualização e mapear cada alvo para seu conjunto de oito LEDs.

7.2. Conflito potencial de pinos

Os pinos D5, D6 e D7 aparecem tanto no vetor PIN_LEDS quanto nas definições de DATA, CLOCK e LATCH do 74HC595. Com USE_74HC595=false, o firmware usa a configuração direta. Na integração com os registradores, o mapeamento deve ser reorganizado para que um mesmo pino não tenha duas funções incompatíveis.

7.3. Calibração dos sensores

O código de teste usa variação positiva (delta), enquanto o firmware principal usa limiar absoluto. Os parâmetros 20 e 100 têm significados diferentes e não devem ser comparados como se fossem a mesma escala. Os limiares definitivos devem ser ajustados com os sensores instalados nas placas e com a estrutura mecânica real.

7.4. Segurança da parada de emergência

O firmware desativa o alvo, apaga os LEDs, emite alarme e envia um evento. Isso interrompe a lógica do treinamento, mas não substitui um circuito físico de segurança para remover energia de motores ou de outros atuadores.

8. Fluxo geral de funcionamento

1. Inicialização: o Arduino configura sensores, LEDs, buzzer e botão de emergência.
2. Autoteste: os LEDs são acionados rapidamente e o buzzer sinaliza que o sistema está pronto.
3. Recebimento do treinamento: o computador envia START com o alvo e o tempo.
4. Indicação do alvo: o Arduino acende o indicador correspondente.
5. Detecção do impacto: o piezo transforma a vibração em sinal elétrico e o Arduino realiza a leitura.
6. Análise: o firmware classifica o resultado como acerto, alvo incorreto ou tempo esgotado.
7. Feedback: LEDs e buzzer informam o resultado ao jogador.
8. Registro: eventos são enviados pela serial para a aplicação externa.
9. Finalização ou emergência: o desafio termina normalmente ou é interrompido pela rotina de emergência.

9. Vídeo do funcionamento

Link do vídeo

10. Conclusão

O protótipo combina aquisição de sinais analógicos, processamento embarcado, indicação visual e sonora e comunicação serial. O código de teste permite validar os sensores separadamente, enquanto o firmware principal organiza o treinamento por estados e eventos. Para a versão final, é essencial concluir a integração entre o controle dos LEDs e os registradores 74HC595, calibrar os limiares dos piezos na estrutura real e validar a parada de emergência também no circuito físico.